ADVANCED HTML & CSS

Full Stack JavaScript learning path

BUILD BEAUTIFUL, ACCESSIBLE, RESPONSIVE WEBSITES

Animation • Accessibility • Responsive Design • Homepage Project

Course roadmap

Three core areas and one practical project

01 Animation

Transforms, transitions, keyframes and movement that communicates meaning.

02 Accessibility

Semantic HTML, WCAG, keyboard navigation, contrast and auditing.

03 Responsive Design

Fluid layouts, responsive images, media queries and a portfolio homepage.

Transforms: change appearance without changing layout

CSS transform can move, rotate and scale an element

```
.card:hover {
  transform: translateY(-8px) scale(1.02);
}
```

Common functions

translate() • rotate() • scale() • skew()

Useful for hover states, icons and micro-interactions; avoid moving important content unexpectedly.

Transitions: smooth changes between states

Set a property, duration and easing

```
button {
 background:#2563eb;
 transition:background .25s ease,
        transform .25s ease;
}
button:hover{transform:translateY(-3px);}
```

Design advice

Animate transform and opacity when possible. Keep motion subtle and purposeful.

Keyframes: build multi-stage animation

@keyframes defines intermediate animation steps

```
@keyframes fadeIn {
  from {opacity:0;transform:translateY(12px);}
  to   {opacity:1;transform:translateY(0);}
}
.hero {animation:fadeIn .6s ease both;}
```

Motion safety

Respect reduced-motion preferences. Movement should never block reading or navigation.

```
@media (prefers-reduced-motion:reduce){
  .hero {animation:none;}
}
```

Accessibility: websites must work for everyone

Accessibility supports different abilities, devices and input methods

Semantic structure

Use real headings, main, nav, labels and buttons so assistive technology understands the page.

Keyboard support

All actions should work with Tab, Enter and Space. Focus indicators must remain visible.

Accessibility is a design requirement - not a feature added at the end.

WCAG and clear content

Four principles: Perceivable, Operable, Understandable, Robust

Readable contrast

Keep text legible against its background.

Meaningful text

Use descriptive links instead of “click here”.

Images and labels

Provide useful alt text and form labels.

Clear navigation

Maintain a predictable heading order and visible focus.

Semantic HTML and accessible interaction

Choose elements based on meaning - not looks

```
<nav aria-label="Main navigation">
  <a href="/">Home</a>
  <a href="/projects">Projects</a>
</nav>
<main>
  <h1>My portfolio</h1>
  <button type="button">See projects</button>
</main>
```

Remember

Prefer native HTML before ARIA. Give every interactive element a clear accessible name.

Responsive layouts: one site, many screen sizes

Start with naturally flexible content

```
.container{width:min(92%,1100px);margin:auto;}
.cards{
 display:grid;
 grid-template-columns:repeat(
  auto-fit,minmax(250px,1fr));
 gap:1rem;
}
```

Core idea

Avoid fixed widths. Let content adapt to available space.

Fluid units • Flexible grids • Relative sizing • Mobile-friendly spacing

Media queries and responsive images

Adapt layout rules and image resources

```
@media (max-width:700px){
 header{flex-direction:column;}
}

<img src="photo.jpg"
   srcset="photo-small.jpg 480w, photo.jpg 1200w"
   sizes="(max-width:700px) 100vw, 50vw"
   alt="Student working on a website">
```

Checklist

Test small and wide screens. Avoid horizontal overflow. Use descriptive alt text. Choose appropriate image sizes.

Final project: responsive homepage

Apply animation, accessibility and responsive design in one portfolio

Plan

Sketch a mobile-first homepage with header, introduction, project cards and contact links.

Build

Use semantic HTML, Grid/Flexbox and fluid image sizes. Add gentle hover effects.

Test

Check keyboard focus, contrast, screen sizes and reduced-motion settings.

Key takeaways & references

From attractive interfaces to inclusive web experiences

- ✓ Motion should clarify, not distract.
- ✓ Semantic markup and keyboard access matter.
- ✓ Responsive layouts adapt instead of breaking.
- ✓ The Homepage project brings everything together.

Course reference: The Odin Project — Advanced HTML and CSS

https://www.theodinproject.com/paths/full-stack-javascript/courses/advanced-html-and-css